

Importing essential resources

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow import keras
from tensorflow.keras import layers
```

Setting paths to the dataset

```
data_train_path = "/content/drive/MyDrive/Deep learning IBM/Fruits_Vegetables/train"
data_test_path = "/content/drive/MyDrive/Deep learning IBM/Fruits_Vegetables/test"
data_val_path = "/content/drive/MyDrive/Deep learning IBM/Fruits_Vegetables/validation"
```

Scaling the Images

```
img_width = 180
img_height = 180
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

Reading the data from the directory to the datasets

```
data_train = tf.keras.utils.image_dataset_from_directory(
    data_train_path,
    shuffle = True,
    image_size = (img_height, img_width),
    batch_size = 32,
    validation_split = False
)
```

Found 3115 files belonging to 36 classes.

```
data_cat = data_train.class_names
```

```
data_cat
```

```
['apple',
 'banana',
 'beetroot',
 'bell pepper',
 'cabbage',
 'capsicum',
 'carrot',
 'cauliflower',
 'chilli pepper',
 'corn',
 'cucumber',
 'eggplant',
 'garlic',
 'ginger',
 'grapes',
 'jalapeno',
 'kiwi',
 'lemon',
 'lettuce',
 'mango',
 'onion',
 'orange',
 'paprika',
 'pear',
 'peas',
 'pineapple',
 'pomegranate',
 'potato',
 'raddish',
 'soy beans',
 'spinach',
 'sweetcorn',
 'sweetpotato',
 'tomato',
```

```
'turnip',
'watermelon']
```

```
data_val = tf.keras.utils.image_dataset_from_directory(
    data_val_path,
    shuffle = False,
    image_size = (img_height, img_width),
    batch_size = 32,
    validation_split=False
)
```

Found 351 files belonging to 36 classes.

```
data_test = tf.keras.utils.image_dataset_from_directory(
    data_test_path,
    shuffle = False,
    image_size = (img_height, img_width),
    batch_size = 32,
    validation_split = False
)
```

Found 359 files belonging to 36 classes.

Printing the Images form the Trainnig dataset

```
plt.figure(figsize=(10,10))
for image, labels in data_train.take(1):
    for i in range(9):
        plt.subplot(3,3,i+1)
        plt.imshow(image[i].numpy().astype('uint8'))
        plt.title(data_cat[labels[i]])
        plt.axis('off')
```

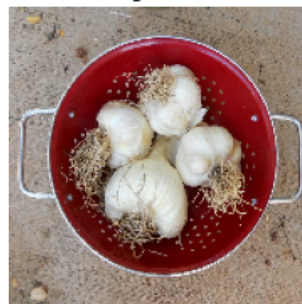
soy beans



pomegranate



garlic



ginger



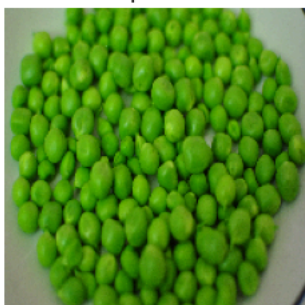
kiwi



grapes



peas



spinach



tomato



Making Sequential Model

- Importing the sequential model the tensor's keras model

```
from tensorflow.keras.models import Sequential
```

```
data_train
```

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 180, 180, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None,), dtype=tf.int32, name=None))>
```

```
model = Sequential([
    layers.Rescaling(1./255),
    layers.Conv2D(16,3,padding = 'same', activation='relu' ),
    layers.MaxPool2D(),
    layers.Conv2D(32,3,padding = 'same', activation='relu' ),
    layers.MaxPool2D(),
    layers.Conv2D(64,3,padding = 'same', activation='relu' ),
    layers.MaxPool2D(),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dropout(0.2),
    layers.Dense(128),
    layers.Dense(len(data_cat))
])
```

```
model.compile(optimizer='adam',loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),metrics=['accuracy'])
```

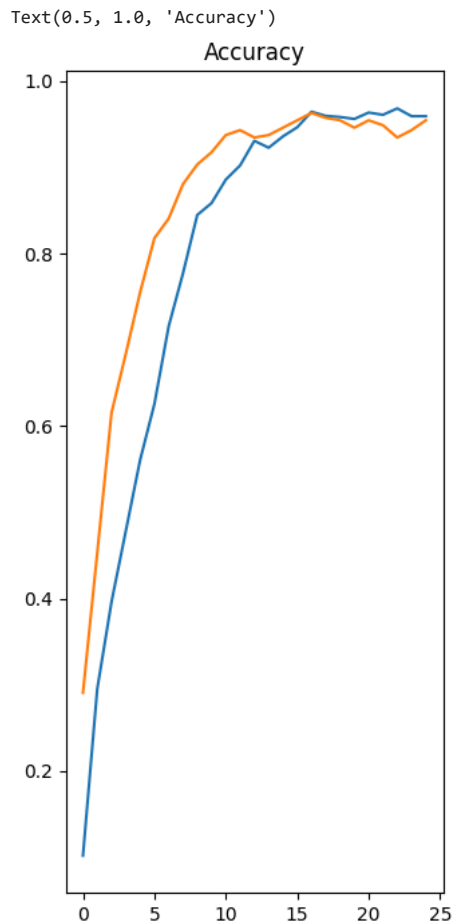
```
epochs_size = 25
```

```
history = model.fit(data_train,validation_data=data_val,epochs=epochs_size)
```

```
Epoch 1/25
98/98 ————— 498s 5s/step - accuracy: 0.0989 - loss: 3.2429 - val_accuracy: 0.3020 - val_loss: 2.4503
Epoch 2/25
98/98 ————— 200s 2s/step - accuracy: 0.2841 - loss: 2.4515 - val_accuracy: 0.4729 - val_loss: 1.8232
Epoch 3/25
98/98 ————— 201s 2s/step - accuracy: 0.3766 - loss: 2.0998 - val_accuracy: 0.6182 - val_loss: 1.3548
Epoch 4/25
98/98 ————— 192s 2s/step - accuracy: 0.4658 - loss: 1.7958 - val_accuracy: 0.6752 - val_loss: 1.2726
Epoch 5/25
98/98 ————— 193s 2s/step - accuracy: 0.5554 - loss: 1.4980 - val_accuracy: 0.7977 - val_loss: 0.7739
Epoch 6/25
98/98 ————— 213s 2s/step - accuracy: 0.6469 - loss: 1.2244 - val_accuracy: 0.8604 - val_loss: 0.5408
Epoch 7/25
98/98 ————— 195s 2s/step - accuracy: 0.7136 - loss: 0.9729 - val_accuracy: 0.8746 - val_loss: 0.4865
Epoch 8/25
98/98 ————— 196s 2s/step - accuracy: 0.7743 - loss: 0.7482 - val_accuracy: 0.9202 - val_loss: 0.3734
Epoch 9/25
98/98 ————— 205s 2s/step - accuracy: 0.8180 - loss: 0.6069 - val_accuracy: 0.9316 - val_loss: 0.3516
Epoch 10/25
98/98 ————— 204s 2s/step - accuracy: 0.8526 - loss: 0.4922 - val_accuracy: 0.9259 - val_loss: 0.3369
Epoch 11/25
98/98 ————— 186s 2s/step - accuracy: 0.8831 - loss: 0.3776 - val_accuracy: 0.9402 - val_loss: 0.3383
Epoch 12/25
98/98 ————— 219s 2s/step - accuracy: 0.8941 - loss: 0.3514 - val_accuracy: 0.9487 - val_loss: 0.2845
Epoch 13/25
98/98 ————— 183s 2s/step - accuracy: 0.9207 - loss: 0.2684 - val_accuracy: 0.9345 - val_loss: 0.3170
Epoch 14/25
98/98 ————— 204s 2s/step - accuracy: 0.9300 - loss: 0.2348 - val_accuracy: 0.9459 - val_loss: 0.3992
Epoch 15/25
98/98 ————— 202s 2s/step - accuracy: 0.9441 - loss: 0.2093 - val_accuracy: 0.9573 - val_loss: 0.3554
Epoch 16/25
98/98 ————— 187s 2s/step - accuracy: 0.9490 - loss: 0.1799 - val_accuracy: 0.9516 - val_loss: 0.3217
Epoch 17/25
98/98 ————— 205s 2s/step - accuracy: 0.9323 - loss: 0.2119 - val_accuracy: 0.9544 - val_loss: 0.2904
Epoch 18/25
98/98 ————— 206s 2s/step - accuracy: 0.9599 - loss: 0.1519 - val_accuracy: 0.9573 - val_loss: 0.3774
Epoch 19/25
98/98 ————— 200s 2s/step - accuracy: 0.9644 - loss: 0.1311 - val_accuracy: 0.9459 - val_loss: 0.3632
Epoch 20/25
98/98 ————— 194s 2s/step - accuracy: 0.9644 - loss: 0.1394 - val_accuracy: 0.9630 - val_loss: 0.3229
Epoch 21/25
98/98 ————— 220s 2s/step - accuracy: 0.9701 - loss: 0.1169 - val_accuracy: 0.9402 - val_loss: 0.3857
Epoch 22/25
98/98 ————— 189s 2s/step - accuracy: 0.9746 - loss: 0.1138 - val_accuracy: 0.9630 - val_loss: 0.3051
Epoch 23/25
98/98 ————— 190s 2s/step - accuracy: 0.9759 - loss: 0.0925 - val_accuracy: 0.9601 - val_loss: 0.3005
Epoch 24/25
98/98 ————— 207s 2s/step - accuracy: 0.9759 - loss: 0.0918 - val_accuracy: 0.9601 - val_loss: 0.3092
Epoch 25/25
98/98 ————— 203s 2s/step - accuracy: 0.9570 - loss: 0.1617 - val_accuracy: 0.9573 - val_loss: 0.3157
```

Accuracy Graph

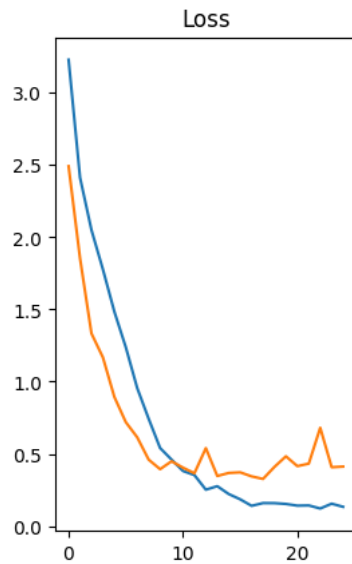
```
epochs_range = range(epochs_size)
plt.figure(figsize=(8,8))
plt.subplot(1,2,2)
plt.plot(epochs_range,history.history['accuracy'],label='Training Accuracy')
plt.plot(epochs_range,history.history['val_accuracy'],label='Validation Accuracy')
plt.title('Accuracy')
```



Loss graph

```
plt.subplot(1,2,2)
plt.plot(epochs_range,history.history['loss'],label='Training loss')
plt.plot(epochs_range,history.history['val_loss'],label='Validation loss')
plt.title('Loss')
```

Text(0.5, 1.0, 'Loss')



```
image = '/content/drive/MyDrive/Deep learning IBM/Corn.jpg'  
image = tf.keras.utils.load_img(image,target_size=(img_height,img_width))  
img_arr = tf.keras.utils.array_to_img(image)  
img_bat = tf.expand_dims(img_arr,0)
```

```
predict = model.predict(img_bat)
```

```
1/1 ————— 0s 166ms/step
```

```
score = tf.nn.softmax(predict)
```

```
print("Veg.fruit in image is {} with accuracy of {:.2f}".format(data_cat[np.argmax(score)],np.max(score)*100))
```

```
Veg.fruit in image is sweetcorn with accuracy of 99.06
```

WebApplication